

10.2 Multivariate Regression



Core skills covered in this section:

- ▶ Mastering the algebraic structure of bivariate and multivariate linear regression models.
- ▶ Understanding, from a geometric perspective, how the regression plane and hyperplane represent relationships between variables.
- ▶ The structure of the design matrix, including the constant column and the columns for each feature.
- ▶ Understanding multivariate regression from the perspective of orthogonal projection.
- ▶ The principle of least squares fitting.
- ▶ Using QR decomposition to solve regression problems.

In the previous section, we analyzed univariate linear regression using orthogonal projection.

In this section, we continue raising the dimension, turning our attention to bivariate linear regression and multivariate linear regression!

⚠ Note that we strongly recommend reading this section alongside the previous section.

Bivariate Linear Regression

Having covered univariate linear regression, let's now raise the dimension and look at bivariate linear regression!

Bivariate linear regression is used to build a linear relationship model between a dependent variable y and two independent variables (x_1, x_2).

As Figure 1 shows, the expression for bivariate linear regression is as follows:

$$y = b_0 + b_1x_1 + b_2x_2 + \varepsilon$$

Here, b_0 is the intercept term, b_1 and b_2 represent the independent-variable coefficients, and ε is the residual term.

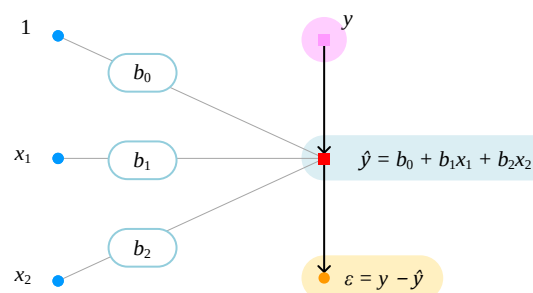


Figure 1. Variable Relationship in Bivariate Linear Regression

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.

Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>

Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

A Plane

From a geometric point of view, bivariate linear regression produces a plane in three-dimensional space, $\hat{y} = b_0 + b_1x_1 + b_2x_2$.

Figure 2 shows the relationship between the scatter points and the regression plane in bivariate linear regression.

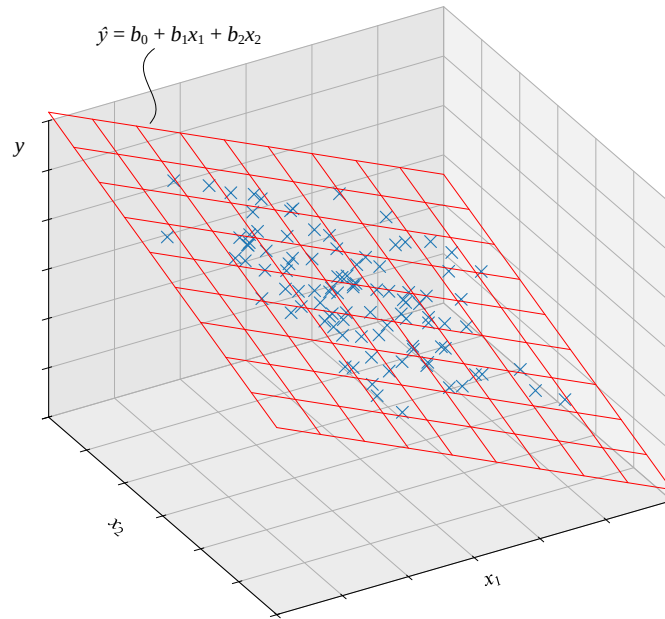


Figure 2. The Plane Corresponding to Bivariate Linear Regression

Next, let's see how the parameters b_0 , b_1 , and b_2 affect the plane $\hat{y} = b_0 + b_1x_1 + b_2x_2$.

b_0 is the intercept term; b_0 controls the overall height of the plane, that is, it shifts the entire plane up or down along the vertical direction, without changing the plane's tilt direction.

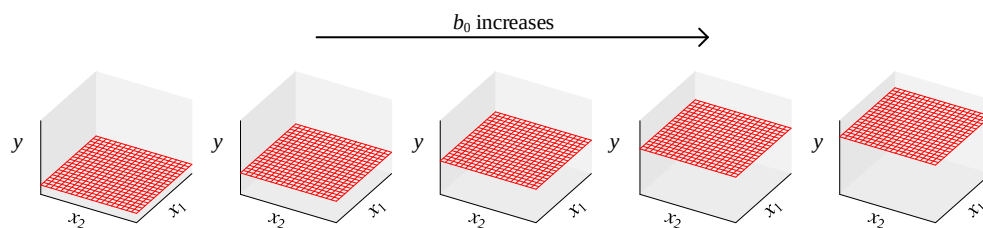


Figure 3. b_0 Controls the Overall Height of the Plane $\hat{y} = b_0 + b_1x_1 + b_2x_2$, $b_1 = 0$, $b_2 = 0$

b_1 and b_2 are the slope coefficients.

The *absolute value* of b_1 controls the slope in the x_1 direction; the sign of b_1 determines whether the plane rises or falls as x_1 increases.

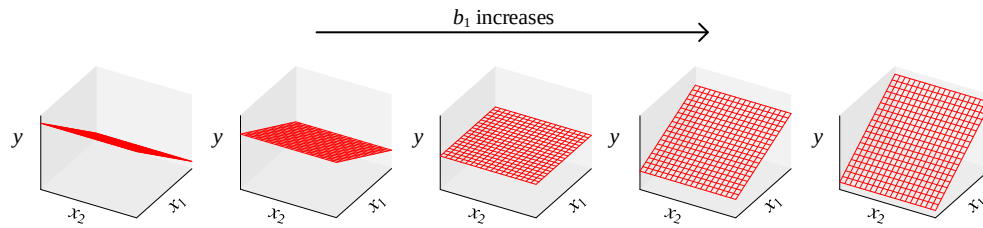


Figure 4. b_1 Controls the Slope of the Plane $\hat{y} = b_0 + b_1x_1 + b_2x_2$ Along the x_1 Direction, $b_0 = 0, b_2 = 0$

The *absolute value* of b_2 then controls the plane's slope in the x_2 direction; the sign of b_2 determines whether the plane rises or falls as x_2 increases.

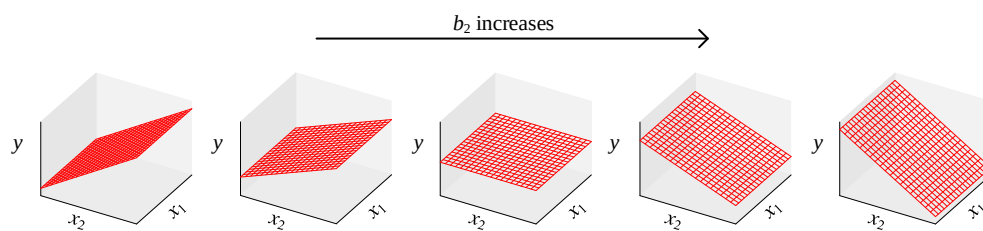


Figure 5. b_2 Controls the Slope of the Plane $\hat{y} = b_0 + b_1x_1 + b_2x_2$ Along the x_2 Direction, $b_0 = 0, b_1 = 0$

Therefore, b_1 and b_2 together determine the plane's "tilt direction" and "steepness," while b_0 determines its vertical position in space.

Bivariate linear regression is about finding suitable b_0 , b_1 , and b_2 so that the plane in Figure 2 fits the data scatter points as closely as possible.

As Figure 1 shows, $\hat{y} = b_0 + b_1x_1 + b_2x_2$ can also be viewed as the superposition of three planes. These three planes are, respectively: a) $\hat{y} = b_0$; b) $\hat{y} = b_1x_1$; c) $\hat{y} = b_2x_2$.

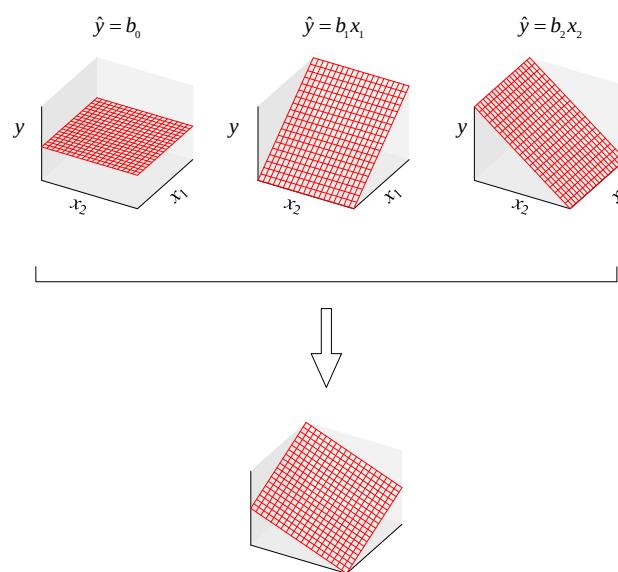


Figure 6. Three Planes Superimposed to Give $\hat{y} = b_0 + b_1x_1 + b_2x_2$

Let's likewise look at bivariate linear regression from the orthogonal-projection perspective introduced in the previous section.

Orthogonal Projection

In bivariate linear regression, the true-value column vector $\mathbf{y}$ can be written as

$$\mathbf{y} = \hat{\mathbf{y}} + \boldsymbol{\varepsilon} = b_0 \mathbf{1} + b_1 \mathbf{x}_1 + b_2 \mathbf{x}_2 + \boldsymbol{\varepsilon}$$

As Figure 7 shows, $\hat{\mathbf{y}}$ is $\mathbf{y}$'s projection in the plane spanned by $\mathbf{1}$, $\mathbf{x}_1$, and $\mathbf{x}_2$, i.e., $\text{span}(\mathbf{1}, \mathbf{x}_1, \mathbf{x}_2)$.

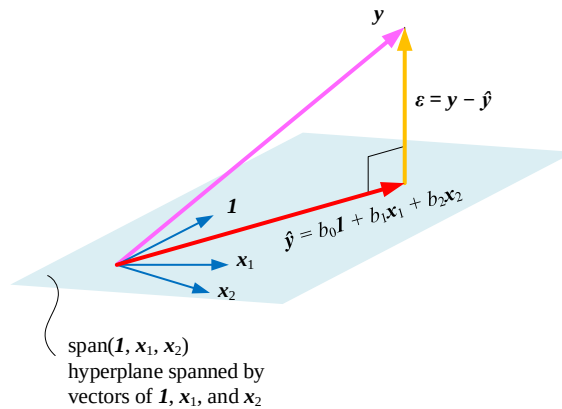


Figure 7. $\hat{\mathbf{y}}$ Projected onto $\text{span}(\mathbf{1}, \mathbf{x}_1, \mathbf{x}_2)$, Bivariate Linear Regression

That is to say, $\hat{\mathbf{y}}$ is a linear combination of the three vectors $\mathbf{1}$, $\mathbf{x}_1$, and $\mathbf{x}_2$

$$\hat{\mathbf{y}} = b_0 \mathbf{1} + b_1 \mathbf{x}_1 + b_2 \mathbf{x}_2$$

Figure 8 shows the shape of the design matrix $\mathbf{X}$ in bivariate linear regression. Bivariate linear regression has two variables, so $\mathbf{X}$ has three column vectors.

⚠ Note that here we assume $\mathbf{1}$, $\mathbf{x}_1$, and $\mathbf{x}_2$ are linearly independent; this way, the design matrix $\mathbf{X}$ has full column rank, and the Gram matrix $\mathbf{X}^T \mathbf{X}$ has full rank; therefore, $\mathbf{X}^T \mathbf{X}$ is invertible. In practice, the column vectors can be linearly dependent. This kind of linear regression requires special treatment.

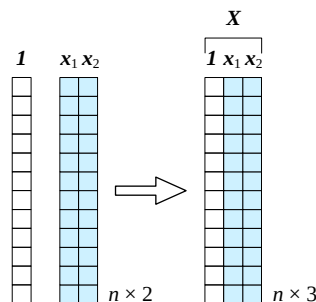


Figure 8. Design Matrix $\mathbf{X}$, Bivariate Linear Regression

Writing the equation above in matrix multiplication form

$$\hat{y} = \underbrace{\begin{bmatrix} 1 & x_1 & x_2 \end{bmatrix}}_X \underbrace{\begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix}}_b = Xb$$

This corresponds to Figure 9.

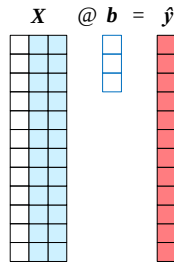


Figure 9. Matrix Operation Giving $\hat{y}$, Bivariate Linear Regression

Figure 10 shows how to compute b in bivariate linear regression.

? Please write out the matrix operation corresponding to Figure 10, and based on the preceding content, derive the matrix multiplication in Figure 10.

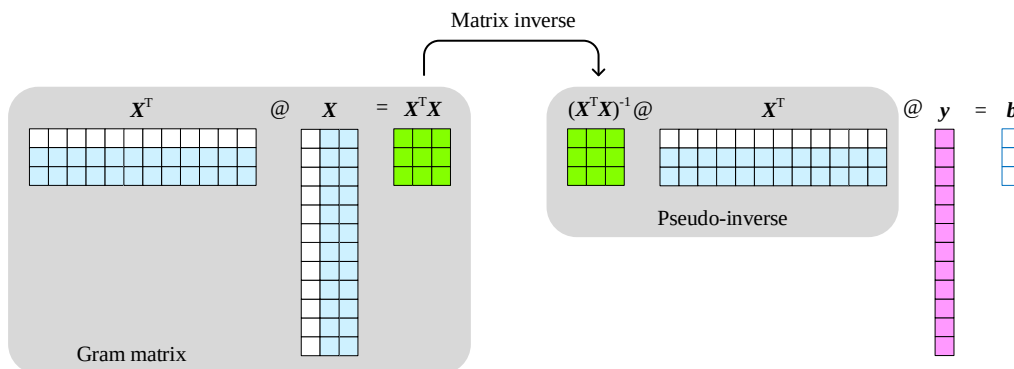
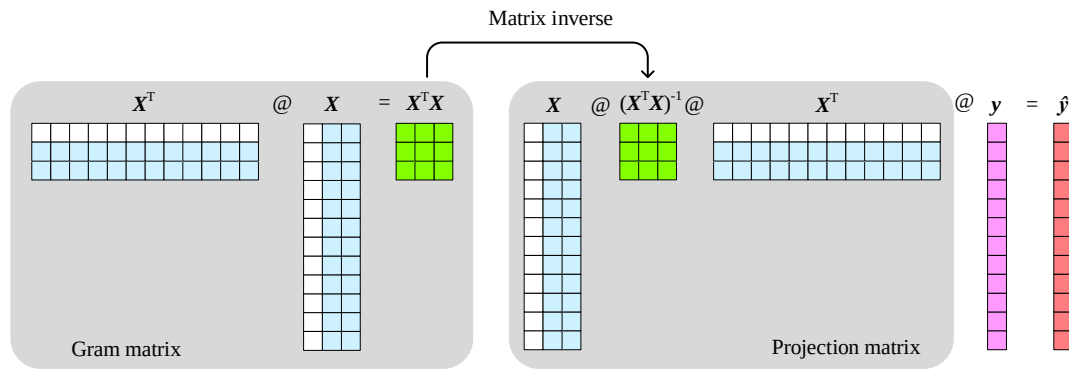


Figure 10. Computing b , Bivariate Linear Regression

Figure 11 shows how to compute $\hat{y}$ in bivariate linear regression.

? Please write out the matrix operation corresponding to Figure 11.

Figure 11. Computing $\hat{y}$, Bivariate Linear Regression

Code Implementation of Bivariate Linear Regression

Code 1 completes the construction of a bivariate linear regression model and plots Figure 2.

Let's talk about the key lines of code below.

a Creates the sample data. Here, `np.random.seed(0)` makes the generated random numbers "reproducible," meaning running this code produces the same random data every time, which is convenient for debugging and comparison. `n_samples = 100` sets the number of generated sample points to 100; this variable is the sample count. `np.random.randn(n_samples, 2)` generates a random array with 100 rows and 2 columns, where the numbers are random draws from the standard normal distribution.

The standard normal distribution is a normal distribution with mean 0 and standard deviation 1. The normal distribution is a symmetric, bell-shaped probability distribution that describes the pattern, seen in many natural phenomena, of data clustering around the mean and gradually decreasing on either side.

➔ The normal distribution, also called the Gaussian distribution, is one of the important topics in the "Math Is Not Hard" series. We'll cover it in Probability and Statistics Is Not Hard.

Code 1's last line generates the dependent-variable data. Specifically, y is the x_1 data multiplied by -3, plus the x_2 data multiplied by 2, plus the constant 1, and then plus a bit of random noise (also generated by `np.random.randn()`), simulating the uncertainty found in real-world data.

b Constructs the "design matrix" for the regression computation. We concatenate a column vector of all ones (representing the constant term) with the two columns of sample data x_1 and x_2 , forming a design matrix X with 100 rows and 3 columns. `np.ones_like(y)` generates an all-ones column vector with the same shape as y , and `np.column_stack()` concatenates the arrays column-wise.

c This formula was already covered in the previous section, and is used to solve for the linear regression. This line corresponds to Figure 10.

d Generates the grid data used to plot the mesh surface. Note that we add a column of 1s in front of the grid-point data matrix, constructing it in the same format as the training data, that is, adding the constant term, so it can be used for prediction.

e Uses the computed column vector b to predict the grid points. This line corresponds to Figure 11.

f Plots a 3D scatter plot of the training sample data.

g Draws the regression plane as a wireframe plot, showing how the predicted values change with x_1 and x_2 . The final visualization result is shown in Figure 2.



Please study LA_10_02_02.ipynb on your own. This code uses the LinearRegression tool provided by Scikit-learn to solve the bivariate linear regression problem. The underlying mathematical principle is exactly the same as the least squares method, and the results are also exactly the same as LA_10_02_01.ipynb. Please study them side by side.

Code 1. Bivariate Linear Regression |  LA_10_02_01.ipynb

```

## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 创建数据
a np.random.seed(0)
n_samples = 100
x_12 = np.random.randn(n_samples, 2)
y = -3*x_12[:,0] + 2*x_12[:,1] + 1 + 0.25*np.random.randn(n_samples)

# 构造设计矩阵, 增加全1列向量
b X = np.column_stack((np.ones_like(y), x_12))

## 创建线性回归模型并拟合数据
c b = np.linalg.inv(X.T @ X) @ X.T @ y

## 拟合
# 生成回归平面的数据点
d x1_grid, x2_grid = np.meshgrid(np.linspace(-3, 3, 10),
                                np.linspace(-3, 3, 10))
X_grid = np.column_stack((x1_grid.flatten(), x2_grid.flatten()))
X_grid = np.column_stack((np.ones_like(X_grid[:,0]), X_grid))

# 预测回归平面上的响应变量
e y_pred = X_grid @ b
y_pred = y_pred.reshape(x1_grid.shape)

## 可视化
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
# 绘制三维样本散点
f ax.scatter(x_12[:,0], x_12[:,1], y, marker = 'x')

# 绘制回归平面
g ax.plot_wireframe(x1_grid, x2_grid, y_pred, colors = 'r')

ax.set_xlabel('$x_1$'); ax.set_ylabel('$x_2$')
ax.set_zlabel('$y$')
ax.set_xlim([-3,3]); ax.set_ylim([-3,3])
ax.set_zlim([-15,15])
ax.set_proj_type('ortho'); ax.view_init(azim=-120, elev=30)

```

Multivariate Linear Regression

Let's continue "raising the dimension" and talk about multivariate linear regression!

Multivariate linear regression is used to build a linear relationship model between a dependent variable and multiple independent variables.

As Figure 12 shows, the expression for multivariate linear regression is as follows:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_Dx_D + \varepsilon$$

Here, b_0 is the intercept term, $b_1, b_2, \dots, b_D$ represent the independent-variable coefficients, ε is the residual term, and D is the number of independent variables.

From a geometric point of view, multivariate linear regression produces a hyperplane. A hyperplane is a generalization of a plane in three-dimensional space.

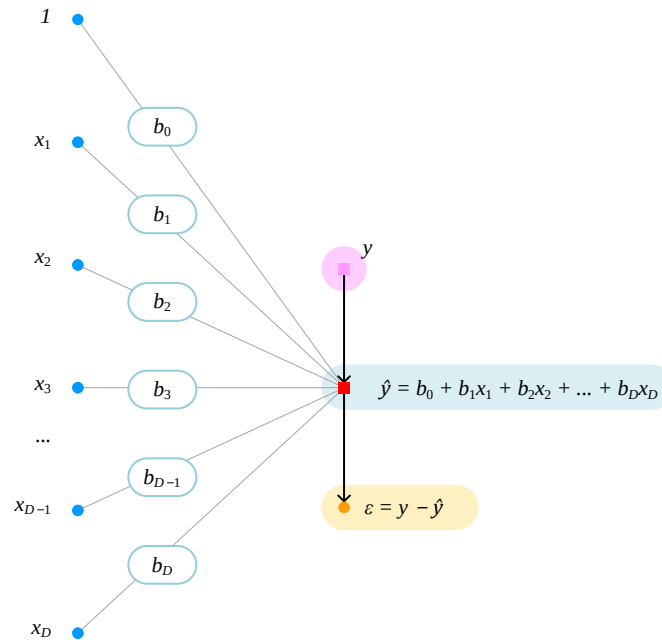


Figure 12. Variable Relationship in Multivariate Linear Regression

Orthogonal Projection

From the perspective of vector spaces, as Figure 13 shows, $\mathbf{1}, \mathbf{x}_1, \mathbf{x}_2, \dots$, and $\mathbf{x}_D$ span $\text{span}(\mathbf{1}, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_D)$.

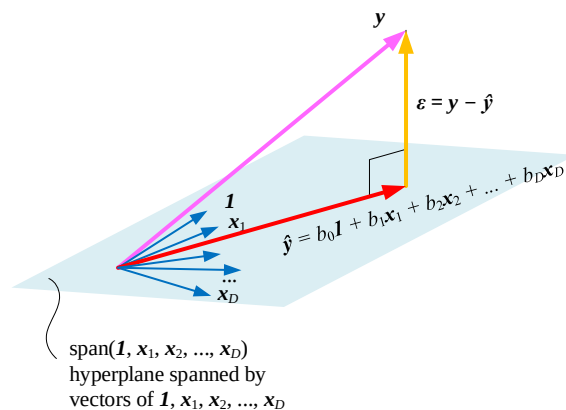


Figure 13. Projecting Vector $\mathbf{y}$ onto the Hyperplane $\text{span}(\mathbf{1}, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_D)$, Multivariate Linear Regression

The result of projecting vector $\mathbf{y}$ onto $\text{span}(\mathbf{1}, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_D)$ is $\hat{\mathbf{y}}$.

That is to say, $\hat{\mathbf{y}}$ can be written as a linear combination of $\mathbf{1}, \mathbf{x}_1, \mathbf{x}_2, \dots$, and $\mathbf{x}_D$

$$\hat{\mathbf{y}} = b_1 \mathbf{x}_1 + b_2 \mathbf{x}_2 + \cdots + b_D \mathbf{x}_D$$

The error $\mathbf{y} - \hat{\mathbf{y}}$ is perpendicular to $\text{span}(\mathbf{1}, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_D)$, that is, $\mathbf{y} - \hat{\mathbf{y}}$ is perpendicular to $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_D$, respectively, that is:

$$\begin{cases} \mathbf{x}_1^T (\mathbf{y} - \hat{\mathbf{y}}) = 0 \\ \mathbf{x}_2^T (\mathbf{y} - \hat{\mathbf{y}}) = 0 \\ \vdots \\ \mathbf{x}_D^T (\mathbf{y} - \hat{\mathbf{y}}) = 0 \end{cases} \Rightarrow \begin{matrix} \mathbf{x}_1^T \\ \mathbf{x}_2^T \\ \vdots \\ \mathbf{x}_D^T \end{matrix} (\mathbf{y} - \hat{\mathbf{y}}) = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

$\mathbf{x}^T$

$\text{span}(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_D)$ and $\text{span}(\boldsymbol{\epsilon})$, i.e., $\text{span}(\mathbf{y} - \hat{\mathbf{y}})$, are mutually orthogonal complements.

Figure 14 shows the design matrix $\mathbf{X}$ for multivariate linear regression, that is,

$$\mathbf{X}_{n \times (D+1)} = [\mathbf{1} \quad \mathbf{x}_1 \quad \mathbf{x}_2 \quad \cdots \quad \mathbf{x}_D]$$

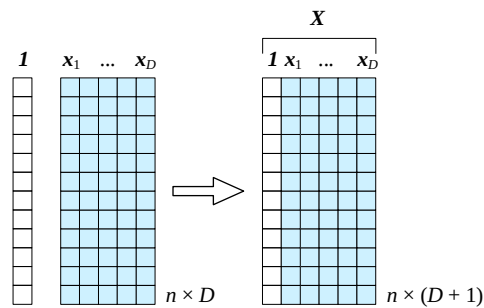


Figure 14. Design Matrix $\mathbf{X}$, Multivariate Linear Regression

Written in Matrix Multiplication Form

The column vector formed by the predicted values, $\hat{\mathbf{y}}$, is computed through the following equation:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{b}$$

The matrix multiplication above corresponds to Figure 15.

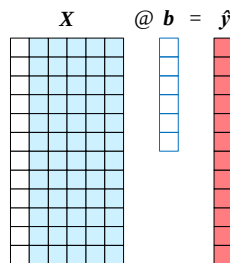


Figure 15. Matrix Operation Giving $\hat{\mathbf{y}}$, Multivariate Linear Regression

Generalized Inverse

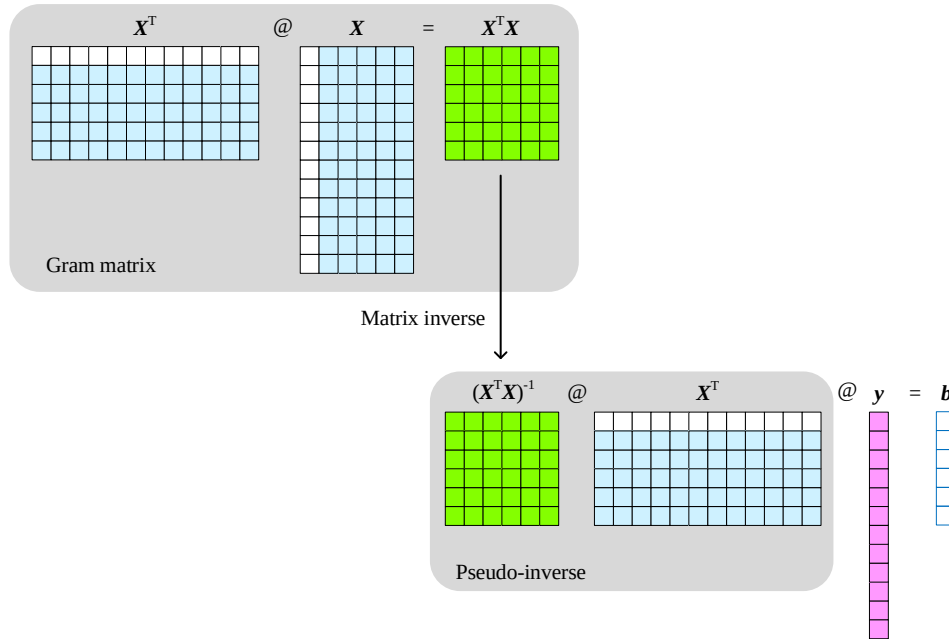
As Figure 16 shows, if $\mathbf{X}^T \mathbf{X}$ is invertible, the solution of $\mathbf{b}$ is

This PDF file is the author's draft, published to make it convenient for readers to study on mobile devices; the final content is subject to the printed edition published by Tsinghua University Press.

Copyright belongs to Tsinghua University Press. Please do not use it commercially, and please credit the source when quoting.
Code and PDF file downloads: <https://github.com/Visualize-ML>

The companion video lectures for this book are all published on Bilibili — 生姜 DrGinger: <https://space.bilibili.com/513194466>
Comments and corrections are welcome. The book's dedicated email address: jiang.visualize.ml@gmail.com

$$\mathbf{b} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Figure 16. Computing $\mathbf{b}$, Multivariate Linear Regression

Overdetermined System of Equations

Actually, solving for $\mathbf{b}$ is equivalent to solving the following overdetermined system of equations

$$\mathbf{X}\mathbf{b} = \mathbf{y}$$

As mentioned earlier in this book, an overdetermined system of equations is one where the number of equations exceeds the number of unknowns.

Multiplying both sides of the equation by $\mathbf{X}^T$ gives

$$\mathbf{X}^T \mathbf{X} \mathbf{b} = \mathbf{X}^T \mathbf{y}$$

And $\mathbf{X}^T \mathbf{X}$ is exactly $\mathbf{X}$'s Gram matrix.

Only when the Gram matrix $\mathbf{X}^T \mathbf{X}$ has full rank ($\mathbf{X}$ has full column rank correspondingly) is $\mathbf{X}^T \mathbf{X}$ invertible. Multiplying both sides by the inverse of $\mathbf{X}^T \mathbf{X}$, we obtain the closed-form least-squares solution for the coefficient vector.

This is solving for $\mathbf{b}$ from the perspective of an overdetermined system of equations.

QR Decomposition

Performing complete QR decomposition on the design matrix $\mathbf{X}$

$$\mathbf{X} = \mathbf{Q}\mathbf{R}$$

$\mathbf{Q}$ is an orthogonal matrix, a square matrix. Writing the equation above as block matrix multiplication

$$X = [Q_1 \quad Q_2] \begin{bmatrix} R_1 \\ O \end{bmatrix}$$

After rearranging, we obtain the reduced QR decomposition

$$X = Q_1 R_1$$

The matrix multiplication can be written as

$$\hat{y} = Q_1 R_1 b$$

Multiplying both sides of the equation above by the semi-orthogonal matrix Q_1^T

$$Q_1^T \hat{y} = Q_1^T Q_1 R_1 b = R_1 b$$

Since R_1 is an upper triangular matrix, computing its inverse is more convenient — this is exactly the advantage of QR decomposition in solving systems of equations.

Projection Matrix

As Figure 17 shows, the fitted-value vector $\hat{y}$ is:

$$\hat{y} = Xb = X(X^T X)^{-1} X^T y$$

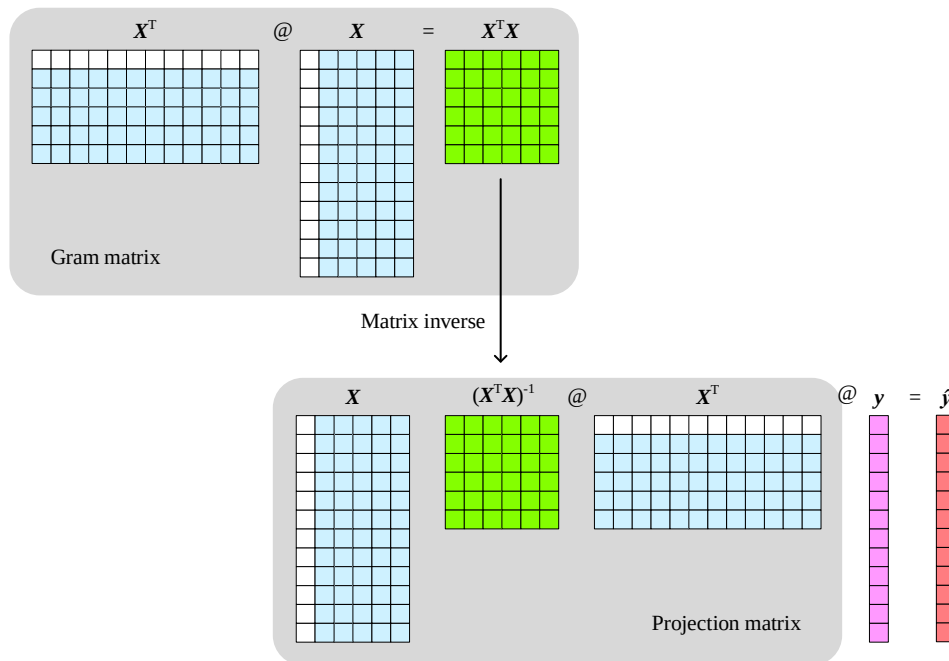


Figure 17. Computing $\hat{y}$, Multivariate Linear Regression

Going back to Figure 13, we notice that in the equation above, $X(X^T X)^{-1} X^T$ is exactly the projection matrix for projecting y onto the plane spanned by X 's column vectors.

The equation above is exactly the vector projection of y onto $\text{span}(1, x_1, x_2, \dots, x_D)$!

As mentioned earlier in this book, $X(X^T X)^{-1} X^T$ is often called the hat matrix.

According to the equation above, the residual column vector $\varepsilon = y - \hat{y}$ can be written as

$$\varepsilon = y - X(X^T X)^{-1} X^T y = (I - X(X^T X)^{-1} X^T) y$$

Going back to Figure 13, we notice that $\text{span}(\varepsilon)$ is the orthogonal complement of $\text{span}(\mathbf{1}, x_1, x_1, \dots, x_D)$!

Looking at it from another angle, ε is exactly the normal vector of the plane $\text{span}(\mathbf{1}, x_1, x_1, \dots, x_D)$.

This means, in the equation above, $(I - X(X^T X)^{-1} X^T)$ is also a projection matrix. This projection matrix projects y onto the "residual" space, giving the residual vector ε .

Adding the two equations above together

$$\hat{y} + \varepsilon = \underbrace{X(X^T X)^{-1} X^T}_{\text{Projection matrix}} y + \underbrace{(I - X(X^T X)^{-1} X^T)}_{\text{Projection matrix}} y = Iy = y$$

Looking at the equation above from the other direction, the vector y is orthogonally decomposed into: the estimated-value vector $\hat{y}$ and the error vector ε .



Please use tools such as DeepSeek/ChatGPT to complete the following exercises in this section.

Q1. The following code generates the independent-variable and dependent-variable data for multivariate regression; please write Python code to compute $\hat{b}$, $\hat{y}$, and ε .

```
## 初始化
import numpy as np

## 创建数据
np.random.seed(0)
n_samples = 100

# 自变量数据
X_vecs = np.random.randn(n_samples, 6)

# 因变量数据
y = X_vecs @ np.array([[-3], [2], [-2], [3], [1], [-1]])
y = y + 1 + 0.25*np.random.randn(n_samples)
```

Q2. Use the LinearRegression function from the sklearn.linear_model module to solve the multivariate linear regression in Q1, and compare the results.